Lab 05 - Queues & Trees

Instructions:

- **A linked full binary tree can be easily generated with the help of a queue, a data structure that follows the First-In-First-Out principle. To ensure that a binary tree is always a full tree, each insert must complete each level in order from left to right before starting the next level. The following algorithm accomplishes that goal in $O(1)$ time complexity:**

```
Insert(T, Q, obj)
01. if T is null and Q.empty(), then
  01. T ← Tnode(obj).
  02. Q.enqueue(T).
02. t ← Q.peek().
03. if t.left not null and t.right not null, then
  01. Q.dequeue().
  02. Q.enqueue(t.left).
  03. Q.enqueue(t.right).
04. if t.left is null, then
  01. t.left ← Tnode(obj).
05. else,
  01. t.right ← Tnode(obj).
```

  where *T* is a tree node and *Q* is a queue.

- **Your objective is to:**
  1. **Copy the *Deque* class from Note10 and convert it to a *Queue* class such that `peek()` is not constant.**
  2. **Define the `Insert()` algorithm.**
  3. **Create a number prediction game.**
  4. **Simulate the game.**

- **You may only modify the files provided. Modifying any included library or the namespace is strictly prohibited.**

- **Documentation for the classes *Node* and *Tnode* is available in the Lab05 directory.**

- **A task will not receive credit if its required prerequisite tasks are not completed.**

- **All submissions must be pushed to the GitHub repository inside the Lab05 directory.**

- **Academic dishonesty of any kind is strictly prohibited.**

- **Any violation of the rules above will result in an automatic grade of zero (0) for the lab.**

Grading

Task	Maximum Points	Points Earned
1	0.5	
2	1.5	
3	1.5	
4	1.5	
Total	5.0	

Task 1

- Within the *dsl* namespace of the '`Queue.h`' header file, copy the *Deque* class from Note10 into it. Afterward, modify it into a *Queue* class that contains:
  - its special member functions.
  - `void enqueue(const T& obj)` - inserts *obj* at the back of the queue.
  - `void dequeue()` - removes the front item from the queue.
  - `T& peek()` - returns the front item of the queue.
  - `bool empty() const` - check if the queue is empty.

Task 2

- Within the *dsl* namespace of the '`FullTree.h`' header file, define the function

```
template <typename T>
void insert(Tnode<T>*& T, Queue<Tnode<T>*>& q, const T& obj)
```

that implement the `insert()` algorithm in the instructions.

Task 3

- Within the *dsl* namespace of the '`BuildGame.h`' header file, define the function

```
Tnode<string>* loadGame(ifstream& fin)
```

that returns a linked full binary tree of the content of the *fin* if *fin* opens; otherwise, an empty tree.

Task 4

- Within the '`main.cpp`' source file, implement a game simulation configured as follows:

Setup Phrase

1. Prompt the user to enter the name of the game file.
2. Create a fstream object with the file name provided.
3. Cretae a *Tnode* pointer object by invoking `loadGame()`.
4. Display the message `"I can guess any number you think of between 1 and 10 with at most 5 questions. Let us begin:\n"`

Simulation Loop While the tree node is not null, perform the following steps:

1. If the data of the node is not an empty string, display it; otherwise, break from the loop.
2. Prompt the user to enter either `'Y'` for yes or `'N'` for no to answer the node question.
3. If the user entered `'Y'`, assign the node its left child; otherwise, asign the node its right child.

Afterward, display the message `"See, I told you\n"` if the last input was `'Y'`; otherwise, display the message `"You are lying\n"`